Data Acquisition Report



ACE

CYBER R&D Program

Conversational AI Interface for
Analyst-Driven Threat Investigation

Koi McFarland
Charleston Southern University
2025

Data Acquisition Report

Document ID:	DAP-001
Version #:	1.1
Status:	Draft
Author:	Koi McFarland
Faculty Advisor:	Prof. Henderson
DoD/ACE Mentor:	Prof. Littleton
Date:	2025-11-20

1.0 Purpose of Data in Research

The synthetic dataset generated in the testbed directly supports the research question and hypothesis by providing a controlled, reproducible environment for evaluating how effectively the Conversational AI Interface (CAI) converts natural-language threat-hunting queries into accurate Elasticsearch JSON DSL. In this study, the dataset is used exclusively for testing, evaluation, and simulation, but not for model training. Python scripts generate both benign and adversarial SOC logs, and each event is labeled using rule-based logic, with ground-truth mappings stored in a separate generated-soc-logs-key.txt file. These labels allow the dataset to serve as the authoritative reference set against which the CAI's performance is measured. The data is aligned with the study's independent variables and supports the primary dependent variable: F1-score, which captures the balance of precision and recall in identifying threat-tagged events. Because each threat category has a corresponding canonical Elasticsearch query derived from the same labeling rules, the dataset enables direct comparisons between CAI-generated queries and ground-truth expected outputs. The expected outcome of using this dataset is the ability to quantitatively determine whether the CAI meets the F1-focused hypothesis criteria, specifically, whether it achieves an overall F1 score of at least 0.95 across all evaluated threat-hunting tasks. The controlled structure and deterministic labeling ensure that any errors can be attributed to the CAI's reasoning or query construction rather than data quality issues, making the dataset essential for rigorous, repeatable, and valid evaluation of the system's effectiveness.

2.0 Data Type & Source

The dataset was generated within a fully controlled environment to ensure that each event could be deterministically labeled, allowing for the creation of an accurate and reliable PI Key for ground-truth validation.

Data Type & Source:

Data Type	Description	Source / Provider	Access Method	Licensing / Ethics
Synthetic	Generated SOC Logs	Python Script	Locally Generated	Internal Use
Synthetic	Log Type Key	Python Script	Locally Generated	Internal Use
Synthetic	Log Generation History	Python Script	Locally Generated	Internal Use

3.0 Data Acquisition / Generation Process

Data for this study was generated entirely within a controlled AWS-based testbed using custom Python log-generation scripts executed on the Workstation EC2 node. The primary tools included Python 3.11, the Faker library, and three specialized modules called BenignLogFaker, ThreatLogFaker, and WeightedIPGenerator, which produced synthetic but realistic ECS-compliant SOC telemetry. The central generator script, generate-logs.py, orchestrated the creation of both benign user-session logs and adversarial threat events, including brute-force login attempts and lateral movement chains. Logs were generated on a scheduled basis using a cron job running every five minutes, producing a consistent volume of timestamped NDJSON entries written to /var/log/generated/soc-generated-logs.ndjson. Each generated event contained structured fields such as @timestamp, host.name, source.ip, event.action, user.name, and tags.

To allow for ground-truth evaluation, the generator simultaneously produced a key mapping file (soc-generated-logs-key.txt) in which every log UUID was paired with an authoritative label identifying its semantic type (e.g., brute_force_failed_login, lateral_movement_rdp). This labeling process is deterministic and derived directly from the adversarial and benign generation logic, ensuring that F1-score calculations and accuracy metrics could be computed with complete precision. Synthetic logs were automatically ingested into Elasticsearch via Filebeat, enabling real-time indexing and making the dataset immediately available for CAI-driven query evaluation. The resulting dataset contains thousands of benign and adversarial events generated over the test period, with each record containing structured features, labels, and microsecond-precision timestamps to support both temporal analysis and comprehensive ground-truth validation.

Data Generation Process:

Step	Description	Tool / Script	Duration	Output
1	Initialize the Weighted IP Generator Loads GeoLite2-City.mmdb and builds five regional IP pools (Charleston, rest of SC, rest of USA, friendly foreign, adversary nations) by sampling the database and counting votes per /16 network.	generate-logs.py ip.py GeoLite2	~3-15 seconds	No direct output, simply creates the IP pools for later use.
2	Parse arguments & calculate targets Reads --min-count, —max-count, and --threat-ratio, creates random count, computes exact number of benign and threat logs needed (e.g., ~150 benign + ~20 threat).	generate-logs.py	< 0.01 seconds	Prints to the console the amount of benign and adversarial logs generated
3	Repeatedly calls BenignLogFaker.generate_session(size=3–6) until target benign count is reached. Each session creates a realistic user session (login → actions → logout) with consistent user ID, session ID, and source IP.	benign.py	< 1 second	No direct console output. Internally produces thousands of log entries that share the same user/session
4	Any shortfall after sessions is filled with individual BenignLogFaker.random_log() calls (one log per call).	benign.py	< 0.1 seconds	Adds miscellaneous standalone events (HTTP requests, process starts, file access, etc.) to hit the exact benign target.
5	Repeatedly runs a custom	threat.py	< 1 second	Creates

Step	Description	Tool / Script	Duration	Output
	threat_campaign() lambda that picks an adversary-heavy IP. Generates a brute-force campaign. Generates a lateral-movement chain to a final host and continues until threat target is reached.			clustered attack chains with the same attacker IP and victim usernames.
6	Any remaining threat slots are filled with single random threat logs (ThreatLogFaker.random_log()).	threat.py	< 0.1 seconds	Adds isolated malicious IP connections, extra RDP/SSH hits, etc.
7	Merges benign + threat lists, sorts every log by its @timestamp.	generate-logs.py	< 0.1 seconds	Ensures chronological order.
8	Writes generated-soc-logs.ndjson and generated-soc-logs-key.txt.	generate-logs.py	< 1 second	generated-soc-logs.ndjson and generated-soc-logs-key.txt.

4.0 Data Characteristics

The synthetic dataset generated for this study consists of time-stamped ECS-compliant SOC log events produced by custom Python scripts and indexed into Elasticsearch via Filebeat. The dataset combines both benign operational activity and clearly defined adversarial behaviors, all labeled automatically through deterministic rule-based logic. The structured format, consistent schema, and microsecond-level timestamps support accurate evaluation of the Conversational AI Interface (CAI) against ground-truth threat indicators.

Data Structure & Properties:

Attribute	Description
Data Format	NDJSON
Number of Records	100-200 documents every 5 minutes; totaling thousands
Number of Features	~40–70 fields per event depending on log type (host, source, destination, user, event, log, network, file, threat, etc.)
Labeled/Unlabeled	Fully labeled using automated deterministic rules; labels stored in soc-generated-logs-key.txt
Time Range	Synthetic timestamps spanning up to 30 days
Sampling Frequency	Log batches generated every 5 minutes via cron automation

Attribute	Description
Data Size	Approximately 31 MB per day of NDJSON logs
Storage Location	Generated on Workstation EC2 (/var/log/generated/), shipped into Elasticsearch indices (soc-logs-YYYY.MM.DD)

5.0 Pre-Processing Steps

Because this study evaluates the accuracy of a Conversational AI Interface (CAI) rather than training a machine-learning model, preprocessing centered on ensuring realistic ECS-compliant log data prior to ingestion and evaluation. The goal of preprocessing was to maintain structural integrity, remove malformed or duplicate records, enforce chronological ordering, and guarantee that each log entry aligned with its corresponding ground-truth label in the PI Key. All transformations were implemented within the log-generation pipeline or within Elasticsearch index settings rather than using ML-style normalization or feature scaling.

Step	Operation	Tool / Library	Purpose	Output
1	Enforce unique UUIDs, remove accidental duplicates	Python (uuid, generator logic)	Ensure clean, non-redundant dataset	One unique event per UUID
2	Chronologically sort logs by timestamp	Python (sorted()) inside generate-logs.py	Preserve temporal realism; required for sequential threat chains	Time-ordered NDJSON
3	Validate ECS-compliant fields	benign.py, threat.py	Guarantee correct indexing and query compatibility	ECS-structured logs
4	Generate deterministic ground-truth labels	Generator writes soc-generated-logs-key.txt	Accurate F1-score and detection evaluation	PI Key with UUID connecting to the Label
5	JSON parsing & discard invalid lines	Filebeat ingest pipeline	Prevent malformed entries in Elasticsearch	Clean, valid JSON events
6	Index partitioning	Elasticsearch	Organize logs by date; improve query speed	soc-logs-YYYY.MM.DD indices

6.0 Data Quality & Validation

Data quality and consistency were validated at multiple stages throughout the generation, ingestion, and evaluation workflow. Because the dataset was produced synthetically under controlled conditions, quality checks focused on (1) verifying structural integrity (ECS compliance, timestamp ordering), (2) ensuring deterministic and correct ground-truth labeling, and (3) confirming that logs were successfully indexed in Elasticsearch without corruption or loss. The unified generator script (GenerateLogs.py) enforces strict UUID uniqueness, complete field population, and chronological ordering, while Filebeat's JSON parser adds additional validation during ingestion. Periodic sampling and query-based verification in Kibana further ensured correctness and internal consistency.

Attribute	Description	Result
Missing Values	Percent of events containing null or missing required fields (@timestamp, log_uuid, host.*, source.ip)	0% (all fields deterministically generated)
Malformed JSON	NDJSON entries rejected by Filebeat JSON decoder	0 observed (generator produces strict JSON)
Duplicate Records	Percent duplicate UUIDs or identical log bodies	0% (unique UUID enforced at generation)
Timestamp Anomalies	Events out of chronological order	0% after sorting; all timestamps validated
Class Imbalance	Ratio of benign to adversarial events	~95% benign / 5% adversarial (intentional threat ratio)
Indexing Errors	Events dropped or failed by Elasticsearch/Filebeat	0 observed
Schema Consistency	Valid ECS fields across dataset	100% compliant for generated fields

7.0 Ethical / Security Considerations

All data used in this study was fully synthetic and generated within a controlled AWS testbed. No real user information, PII, CUI, or operational telemetry was collected or processed. Hostnames, IP addresses, usernames, and other identity-related attributes were produced through Faker-based generators, ensuring all values are non-attributable and non-identifiable. As a result, the dataset is exempt from FERPA, IRB oversight, and DoD restrictions related to handling real-world sensitive information.

Data storage and access adhered to DoD/ACE security expectations. All synthetic logs were stored on encrypted AWS EBS volumes, with access restricted through IAM role-based controls, MFA, SSH key authentication, and least-privilege permissions. Network isolation was maintained using a dedicated VPC, private subnets, and security groups. Log data remained internal to the testbed environment and was not transferred outside controlled systems.

No anonymization procedures were required beyond synthetic generation, as no real identifiers were present. The GitHub repository contained only scripts and documentation and did not store any log data. All research activities were conducted in accordance with secure data handling practices and established DoD cyber research standards.

8.0 Data Sampling & Collection Intervals

Parameter	Description
Sampling Frequency	Log batches generated every 5 minutes via cron scheduling
Events per Batch	100-200 events per 5-minute interval
Daily Volume	28,800–57,600 events per 24-hour period
Collection Duration	Continuous generation across multiple days during the experiment
Temporal Granularity	Microsecond-precision timestamps for all events
Time Window	Full 24-hour coverage; synthetic timestamps span up to 30 days
Total Samples	Thousands of ECS-compliant events (Exact unknown)

9.0 Data Storage & Management

All data used in this study was fully synthetic and generated within a controlled AWS testbed. No real user information, PII, CUI, or operational telemetry was collected or processed. Hostnames, usernames, event fields, and other identity-related attributes were produced through Faker-based generators, ensuring all values are non-attributable and non-identifiable. IP address metadata was additionally enriched using the MaxMind GeoLite2 geolocation database; however, this enrichment was applied only to synthetically generated IPs. No real-world IPs, ASN data, or location data linked to actual individuals or organizations were collected, queried, or stored. Because both the underlying logs and all derived geolocation fields originate from synthetic inputs, the dataset remains free of any personally identifiable or operationally sensitive information. As a result, the data used in this research is exempt from FERPA, IRB oversight, and DoD restrictions related to the handling of real-world sensitive information.

Data storage and access followed DoD/ACE security expectations. All synthetic logs were stored on encrypted AWS EBS volumes, with access restricted through IAM role-based controls, MFA, SSH key authentication, and least-privilege permissions. Network isolation was maintained using a dedicated VPC, private subnets, and security groups. Log data remained entirely internal to the testbed environment and was never transmitted outside approved and controlled systems.

No anonymization procedures were required beyond synthetic generation and GeoLite2 enrichment, as no real identifiers or real-world locations were present in any dataset. The public GitHub repository contained only scripts and documentation and did not store any generated log data. All research activities were conducted in accordance with secure data-handling practices and established DoD cyber research standards.

10.0 Limitations & Known Issues

The primary limitation of this dataset is its reliance on fully synthetic log generation, which, while structurally accurate and ECS-compliant, does not fully capture the complexity, variability, and multi-stage behavior found in real enterprise SOC environments. User activity patterns, host identities, and network interactions are randomly generated rather than reflecting long-term behavioral baselines, organizational hierarchy, or realistic access patterns. As a result, the dataset lacks subtle correlations, rare edge cases, and environmental noise that naturally occur in production ecosystems. Adversarial events, such as brute-force attempts, lateral movement sequences, follow deterministic patterns defined by the generator scripts and may not fully represent the unpredictability or sophistication of real attackers.

11.0 Summary Table

Section	Key Content
Data Type	Synthetic SOC log data (NDJSON, ECS-compliant), including benign events and scripted adversarial behaviors (brute force, lateral movement, IOC matches, C2 beacons).
Source	Fully generated via custom Python scripts (GenerateLogs.py, BenignFaker.py, ThreatFaker.py) executed on the Workstation EC2 instance; no real-world data collected.
Volume	100-200 events every 5 minutes; approximately 28,800–57,600 events per 24-hour period depending on run duration and configuration.
Pre-processing	UUID enforcement, timestamp sorting, ECS schema validation, JSON parsing by Filebeat, and deterministic ground-truth labeling via PI Key (uuid → label).
Validation	Checks for missing fields, malformed JSON, duplicate records,

Section	Key Content
	timestamp consistency, correct label mapping, and ingestion verification through Elasticsearch and random sampling.
Security	All data synthetic (no PII/CUI), stored on encrypted AWS EBS volumes, IAM/MFA-secured access, isolated VPC subnets, and no raw data stored outside the testbed.
Storage	Raw logs stored under /var/log/generated/; indexed in Elasticsearch as soc-logs-YYYY.MM.DD; scripts and documentation maintained in GitHub (no raw data committed).
Use	Used exclusively for evaluating the Conversational AI Interface's query accuracy and detection performance using ground-truth threat labels (F1-score, recall, and detection accuracy).

12.0 Revision History

Version #	Date	Author(s)	Summary of Change
1.1	2025-10-29	Koi McFarland	Filled out document.

13.0 Mentor Review & Approval

Reviewer	Role	Date	Notes
Student Peer	PI	2025-12-17	Final Deliverable
Faculty Advisor			
DoD/ACE Mentor			